

A Technical Introduction to Monte Carlo tree search

Section 1

In computer science, Monte Carlo tree search (MCTS) is a heuristic search algorithm for some kinds of decision processes, most notably those employed in software that plays board games. In that context MCTS is used to solve the game tree. MCTS was combined with neural networks in 2016 and has been used in multiple board games like Chess, Shogi, Checkers, Backgammon, Contract Bridge, Go, Scrabble, and Clobber as well as in turn-based-strategy video games (such as Total War: Rome II's implementation in the high level campaign AI) and applications outside of games.

Section 2

Domain-specific knowledge may be employed when building the game tree to help the exploitation of some variants. One such method assigns nonzero priors to the number of won and played simulations when creating each child node, leading to artificially raised or lowered average win rates that cause the node to be chosen more or less frequently, respectively, in the selection step. A related method, called progressive bias, consists in adding to the UCB1 formula a b_i/n_i element, where b_i is a heuristic score of the i -th move. The basic Monte Carlo tree search collects enough information to find the most promising moves only after many rounds; until then its moves are essentially random. This exploratory phase may be reduced significantly in a certain class of games using RAVE (Rapid Action Value Estimation). In these games, permutations of a sequence of moves lead to the same position. Typically, they are board games in which a move involves placement of a piece or a stone on the board. In such games the value of each move is often only slightly influenced by other moves. In RAVE, for a given game tree node N , its child nodes C_i store not only the statistics of wins in playouts started in node N but also the statistics of wins in all playouts started in node N and below it, if they contain move i (also when the move was played in the tree, between node N and a playout). This way the contents of tree nodes are influenced not only by moves played immediately in a given position but also by the same moves played later.

Section 3

Pure Monte Carlo game search This basic procedure can be applied to any game whose positions necessarily have a finite number of moves and finite length. For each position, all feasible moves are determined: k random games are played out to the very end, and the scores are recorded. The move leading to the best score is chosen. Ties are broken by fair coin flips. **Pure Monte Carlo Game Search**

results in strong play in several games with random elements, as in the game EinStein würfelt nicht!. It converges to optimal play (as k tends to infinity) in board filling games with random turn order, for instance in the game Hex with random turn order. DeepMind's AlphaZero replaces the simulation step with an evaluation based on a neural network.

Section 4

When using RAVE, the selection step selects the node, for which the modified UCB1 formula $(1 - \beta(n_i, n_{\sim i}))w_i/n_i + \beta(n_i, n_{\sim i})w_{\sim i}/n_{\sim i} + c \ln \frac{n_i}{n_{\sim i}}$ has the highest value. In this formula, $w_{\sim i}$ and $n_{\sim i}$ stand for the number of won playouts containing move i and the number of all playouts containing move i , and the $\beta(n_i, n_{\sim i})$ function should be close to one and to zero for relatively small and relatively big n_i and $n_{\sim i}$, respectively. One of many formulas for $\beta(n_i, n_{\sim i})$, proposed by D. Silver, says that in balanced positions one can take $\beta(n_i, n_{\sim i}) = \frac{n_{\sim i}n_i + n_{\sim i} + 4b^2n_i}{n_{\sim i}n_i + n_{\sim i} + 4b^2n_i + 4b^2n_i}$, where b is an empirically chosen constant. Heuristics used in Monte Carlo tree search often require many parameters. There are automated methods to tune the parameters to maximize the win rate. Monte Carlo tree search can be concurrently executed by many threads or processes. There are several fundamentally different methods of its parallel execution:

Section 5

Bibliography Cameron Browne; Edward Powley; Daniel Whitehouse; Simon Lucas; Peter I. Cowling; Philipp Rohlfshagen; Stephen Tavener; Diego Perez; Spyridon Samothrakis; Simon Colton (March 2012). "A Survey of Monte Carlo Tree Search Methods". *IEEE Transactions on Computational Intelligence and AI in Games*. 4 (1): 1–43. Bibcode:2012ITCIA...4A6810B. CiteSeerX 10.1.1.297.3086. doi:10.1109/tciaig.2012.2186810. S2CID 9316331. Maciej Wiechowski, Konrad Godlewski, Bartosz Sawicki, Jacek Małdziuk (July 2022). "Monte Carlo Tree Search: a Review of Recent Modifications and Applications". *Springer Nature Artificial Intelligence Review*. 56 (3): 497–2562 (66 pages). arXiv:2103.04931. doi:10.1007/s10462-022-10228-y. CS1 maint: multiple names: authors list (link)

Section 6

Monte Carlo method The Monte Carlo method, which uses random sampling for deterministic problems which are

difficult or impossible to solve using other approaches, dates back to the 1940s. In his 1987 PhD thesis, Bruce Abramson combined minimax search with an expected-outcome model based on random game playouts to the end, instead of the usual static evaluation function. Abramson said the expected-outcome model "is shown to be precise, accurate, easily estimable, efficiently calculable, and domain-independent." He experimented in-depth with tic-tac-toe and then with machine-generated evaluation functions for Othello and chess. Such methods were then explored and successfully applied to heuristic search in the field of automated theorem proving by W. Ertel, J. Schumann and C. Suttner in 1989, thus improving the exponential search times of uninformed search algorithms such as e.g. breadth-first search, depth-first search or iterative deepening. In 1992, B. Brügmann employed it for the first time in a Go-playing program. In 2002, Chang et al. proposed the idea of "recursive rolling out and backtracking" with "adaptive" sampling choices in their Adaptive Multi-stage Sampling (AMS) algorithm for the model of Markov decision processes. AMS was the first work to explore the idea of UCB-based exploration and exploitation in constructing sampled/simulated (Monte Carlo) trees and was the main seed for UCT (Upper Confidence Trees).